

Lab 12 – Hashing

Task 01:

Suppose that the following character codes are used: 'A' =1, 'B' =2, ..., 'Y' =25, 'Z' = 26. Using a hash table with eleven locations and the hashing function $h(\text{identifier}) = \text{average} \% 11$, where average is the average of the codes of the first and last letters in identifier, show the hash table that results when the following identifiers are inserted in the order given, assuming that collisions are resolved using linear probing: BETA, RATE, FREQ, ALPHA, MEAN, SUM, NUM, BAR, WAGE, PAY, KAPPA.

Write a program for the above problem.

Code:

```
#include<iostream>
#include<string>
using namespace std;

int getAverage(string s){
    int a = s[0] - 'A';
    int b = s[s.length()-1] - 'A';

    return (a+b)/2;
}

int h(int key, int size){ return key % size; }

int prob(int idx, int size, int i){
    return (idx+i) % size;
}

int main(){
    string names[11] = {"BETA", "RATE", "FREQ", "ALPHA", "MEAN", "SUM", "NUM",
"BAR", "WAGE", "PAY", "KAPPA"};

    string hashTable[11];

    // inserting
    for(int i = 0; i < 11; i++){
        int avg = getAverage(names[i]);
        int idx = h(avg, 11);

        if(hashTable[idx] != ""){
            int j = 1;
            int originalIdx = idx;
```

```
        while(true){
            idx = prob(originalIdx, 11, j);

            if(hashTable[idx] == ""){
                hashTable[idx] = names[i];
                break;
            }

            j++;
            if(j == 11){
                cout << "Table is full.\n";
                break;
            }
        }
    } else hashTable[idx] = names[i];
}

// printing
for(int i = 0; i < 11; i++){
    if(hashTable[i] == "-1"){
        cout << "[" << i << "]" -> empty.\n";
    } else cout << "[" << i << "]" -> " << hashTable[i] << "\n";
}

}
```

Output:

```
[0] -> BETA
[1] -> FREQ
[2] -> ALPHA
[3] -> MEAN
[4] -> SUM
[5] -> NUM
[6] -> WAGE
[7] -> KAPPA
[8] -> PAY
[9] -> BAR
[10] -> RATE
```

Task 02:

As a software engineer at **GeoMap Solutions**, you are tasked to design a system that stores country names with respect to their capitals as a key value pair in a hash table according to their lengths.

Write a program and perform following task:

1. Insert given key, value pairs in the hashtable: (United States, Washington DC), (Brazil, Brasília), (Japan, Tokyo), (Germany, Berlin), (Pakistan, Islamabad), (Argentina, Buenos Aires), (Australia, Canberra).
2. Delete a country from the hashtable.
3. Update a capital using its country name.
4. Search for capital with respect to its country name.

Note: Use linear probing for collision handling if there's any.

Code:

```
/* I have misinterpreted the question, the role of country and captail is reversed. */
```

```
#include<iostream>
#include<string>
using namespace std;

#define TABLE_SIZE 10

int myHash(int key) { return key % TABLE_SIZE; }

int prob(int index, int i) { return (index+i) % TABLE_SIZE; }

// 1. Insert
void insert(string hashTable[][2], string country, string capital){
    int idx = myHash(capital.length());

    if(hashTable[idx][0] != ""){
        int j = 1;
        int originalIdx = idx;

        while(j < TABLE_SIZE){
            idx = prob(originalIdx, j);

            if(hashTable[idx][0] == ""){
                hashTable[idx][0] = country;
                hashTable[idx][1] = capital;
                break;
            }
            j++;
        }
    }
}
```

```
    }

    if(j == TABLE_SIZE){
        cout << "Table is full.\n";
    }

} else {
    hashTable[idx][0] = country;
    hashTable[idx][1] = capital;
}
}

// 2. Delete a country from the hash table
bool deleteCountry(string table[][2], string country, string capital){
    int idx = myHash(capital.length());

    if(table[idx][0] != country){
        int j = 1;

        while(j < TABLE_SIZE){
            idx = prob(idx, j);

            if(table[idx][0] == country){
                table[idx][0] = "";
                table[idx][1] = "";
                return true;
            }
            j++;
        }

        if(j == TABLE_SIZE){
            cout << "Country not found in table.\n";
            return false;
        }
    } else {
        table[idx][0] = "";
        table[idx][1] = "";
        return true;
    }
    return false;
}

// 3. Update a capital using its country name
void update(string table[][2], string country, string oldCapital, string
newCapital){
```

```
// first I'll delete, then will re insert it as per capital

    if(deleteCountry(table, country, oldCapital)) insert(table, country,
newCapital);
}

// 4. Search for country by captital
void search(string table[][2], string capital){
    int idx = myHash(capital.length());

    if(table[idx][1] != capital){
        int j = 1;
        while(j < TABLE_SIZE){
            idx = prob(idx, j);

            if(table[idx][1] == capital){
                cout << "Name: " << table[idx][0] << "\n";
                cout << "Capital: " << table[idx][1] << "\n";
                return;
            }
            j++;
        }
        if(j == TABLE_SIZE){
            cout << "Country not found.\n";
            return;
        }
    }
    else{
        cout << "Name: " << table[idx][0] << "\n";
        cout << "Capital: " << table[idx][1] << "\n";
        return;
    }
}

int main(){

    string hashTable[TABLE_SIZE][2];

    // inserting values
    insert(hashTable, "United States", "Washington DC");
    insert(hashTable, "Brazil", "Brasilia");
    insert(hashTable, "Japan", "Tokyo");
    insert(hashTable, "Germany", "Berlin");
    insert(hashTable, "Pakistan", "Islamabad");
    insert(hashTable, "Argentina", "Buenos Aires");
```

```
insert(hashTable, "Australia", "Canberra");

// printing table
cout << "Initial Hash Table:\n\n";

for(int i = 0; i < TABLE_SIZE; i++){

    cout << "[" << i << "]" -> " ";

    if(hashTable[i][0] == ""){
        cout << "empty\n";
    }
    else{
        cout << hashTable[i][0] << " : " << hashTable[i][1] << "\n";
    }
}

// searching
cout << "\nSearching for Japan:\n";
search(hashTable, "Tokyo");

// updating
cout << "\nUpdating capital of Brazil...\n";
update(hashTable, "Brazil", "Brasilia", "Rio de Janeiro");

// deleting
cout << "\nDeleting Germany...\n";

if(deleteCountry(hashTable, "Germany", "Berlin")){
    cout << "Germany deleted successfully.\n";
}

// final table
cout << "\nFinal Hash Table:\n\n";

for(int i = 0; i < TABLE_SIZE; i++){
    cout << "[" << i << "]" -> " ";
    if(hashTable[i][0] == ""){
        cout << "empty\n";
    }
    else{
        cout << hashTable[i][0] << " : " << hashTable[i][1] << "\n";
    }
}
}
```

Output:

```
Initial Hash Table:

[0] -> Australia : Canberra
[1] -> empty
[2] -> Argentina : Buenos Aires
[3] -> United States : Washington DC
[4] -> empty
[5] -> Japan : Tokyo
[6] -> Germany : Berlin
[7] -> empty
[8] -> Brazil : Brasilia
[9] -> Pakistan : Islamabad

Searching for Japan:
Name: Japan
Capital: Tokyo

Updating capital of Brazil...

Deleting Germany...
Germany deleted successfully.

Final Hash Table:

[0] -> Australia : Canberra
[1] -> empty
[2] -> Argentina : Buenos Aires
[3] -> United States : Washington DC
[4] -> Brazil : Rio de Janeiro
[5] -> Japan : Tokyo
[6] -> empty
[7] -> empty
[8] -> empty
[9] -> Pakistan : Islamabad
PS C:\Users\Muhammad Hasan\OneDrive\DSA\University\Lab 12 - Hashing>
```

Activate Windows
Go to Settings to activate Windows.

Task 03:

You are required to implement a hash table for managing a store's fruit inventory system. The hash table will store fruit names and resolve collisions using quadratic probing. The hash function for placing fruit names in the table will be based on the length of the fruit's name, evaluated as $\text{length of fruit name} \% 11$, with the table size fixed at 11.

The program will support four operations: inserting a fruit, searching for a fruit by name, deleting a fruit, and printing the current hash table contents.

For collisions, quadratic probing is used, where the next slot is determined by the formula $(\text{index} + i^2) \% 11$, incrementing i for each subsequent collision.

Insert fruits:

("Apple"), ("Banana"), ("Grapes"), ("Mango"), ("Peach")

Code:

```
#include<iostream>
using namespace std;

#define TABLE_SIZE 11

struct Fruit{
    string name;
    float price;
};

int myHash(string key){
    return (key.length() % TABLE_SIZE);
}

int prob(int hashIndex, int i){
    return ((hashIndex + (i*i)) % TABLE_SIZE);
}

void insertFruit(Fruit table[], Fruit f){
    int idx = myHash(f.name);

    if(table[idx].name != ""){
        int j = 1;
        int originalIdx = idx;

        while(j < TABLE_SIZE){
            idx = prob(originalIdx, j);

            if(table[idx].name == ""){
```



```
        table[idx] = f;
        return;
    }
    j++;
}

if(j == TABLE_SIZE){
    cout << "Table is full.\n";
    return;
}

} else table[idx] = f;
}

int main(){
    Fruit fruitTable[TABLE_SIZE] = {}; // the empty curly braces will initialize
it with: "" & 0.00

    // printing initial state
    for(int i = 0; i < TABLE_SIZE; i++){
        if(fruitTable[i].name == ""){
            cout << "[" << i << "]" -> empty.\n";
        } else cout << "[" << i << "]" -> " << fruitTable[i].name << ", " <<
fruitTable[i].price << "\n";
        }

    insertFruit(fruitTable, {"Mango", 100.0});
    insertFruit(fruitTable, {"Apple", 100.0});
    insertFruit(fruitTable, {"Orange", 100.0});
    insertFruit(fruitTable, {"Banana", 100.0});
    insertFruit(fruitTable, {"Water Melon", 100.0});

    cout << "\n";
    for(int i = 0; i < TABLE_SIZE; i++){
        if(fruitTable[i].name == ""){
            cout << "[" << i << "]" -> empty.\n";
        } else cout << "[" << i << "]" -> " << fruitTable[i].name << ", " <<
fruitTable[i].price << "\n";
        }
    }
}
```

Output:

```
[0] -> empty.  
[1] -> empty.  
[2] -> empty.  
[3] -> empty.  
[4] -> empty.  
[5] -> empty.  
[6] -> empty.  
[7] -> empty.  
[8] -> empty.  
[9] -> empty.  
[10] -> empty.  
  
[0] -> Water Melon, 100  
[1] -> empty.  
[2] -> empty.  
[3] -> empty.  
[4] -> empty.  
[5] -> Mango, 100  
[6] -> Apple, 100  
[7] -> Orange, 100  
[8] -> empty.  
[9] -> empty.  
[10] -> Banana, 100
```